

Manual Técnico

Sistema AFIP Mobile – Gestión Financiera Personal

Versión: 1.0.0

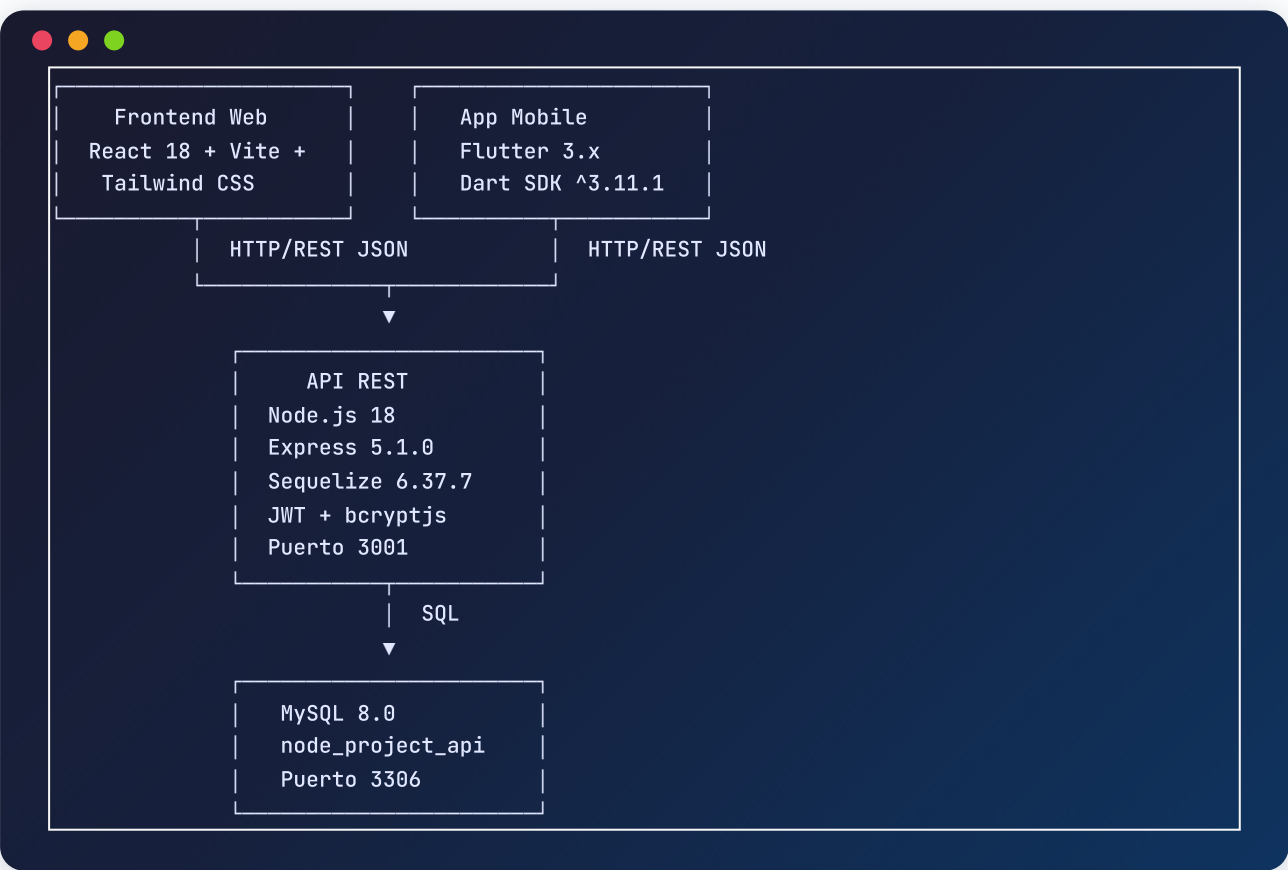
Fecha: Junio 2026

Tipo: Documento Técnico Interno

Audiencia: Desarrolladores Backend, Frontend, Mobile y DevOps

1. Resumen del Sistema

AFIP Mobile es una plataforma web y mobile de gestión financiera personal. El sistema sigue una arquitectura de **tres capas** con API REST centralizada:



2. Estructura de la API (Node.js)

2.1 Árbol de Directorios

```
api/Node_api/
├── .env                # Variables de entorno (no en Git)
├── package.json        # Dependencias y scripts
├── migrate.js          # Script de migración
├── migrations/         # Archivos de migración Sequelize
├── src/
│   ├── index.js        # Punto de entrada; inicia el servidor
│   ├── app/
│   │   └── app.js      # Configuración de Express; montaje de routers
│   ├── config/
│   │   ├── connect.db.js # Conexión a MySQL vía Sequelize
│   │   ├── models.app.js # Asociaciones y sincronización de modelos
│   │   └── predefinedCategories.js # Seed de categorías predefinidas
│   ├── controllers/    # Lógica de negocio (9 controllers)
│   ├── middlewares/
│   │   ├── auth.middleware.js # Verificación de JWT
│   │   └── user.middleware.js # Validación de schemas Joi
│   ├── models/         # Modelos Sequelize (10 modelos)
│   ├── routers/        # Endpoints y rutas (9 routers)
│   ├── schemes/        # Schemas de validación Joi (9 schemas)
│   └── utils/
│       └── initializeUserCategories.js # Inicializar categorías al registrar
```

2.2 Dependencias Principales

Paquete	Versión	Propósito
<code>express</code>	<code>^5.1.0</code>	Framework web HTTP
<code>sequelize</code>	<code>^6.37.7</code>	ORM para MySQL
<code>mysql2</code>	<code>^3.15.3</code>	Driver MySQL para Sequelize
<code>jsonwebtoken</code>	<code>^9.0.2</code>	Generación y verificación de JWT
<code>bcryptjs</code>	<code>^3.0.3</code>	Hashing seguro de contraseñas
<code>@hapi/joi</code>	<code>^17.1.1</code>	Validación de schemas de entrada
<code>cors</code>	<code>^2.8.5</code>	Política de CORS
<code>morgan</code>	<code>^1.10.1</code>	Logger HTTP (desarrollo)
<code>dotenv</code>	<code>^17.2.3</code>	Carga de variables de entorno

`nodemon`

^3.1.10

Reinicio automático en dev

3. Variables de Entorno

Archivo: `api/Node_api/.env`

Variable	Tipo	Default	Descripción
<code>SERVER_PORT</code>	Integer	<code>3001</code>	Puerto en que escucha la API
<code>DB_NAME</code>	String	<code>node_project_api</code>	Nombre de la base de datos
<code>DB_HOST</code>	String	<code>127.0.0.1</code>	Hostname del servidor MySQL
<code>DB_USER</code>	String	<code>root</code>	Usuario MySQL
<code>DB_PASSWORD</code>	String	<code>''</code>	Contraseña MySQL (vacía solo en desarrollo)
<code>DB_PORT</code>	Integer	<code>3306</code>	Puerto MySQL
<code>DB_DIALECT</code>	String	<code>mysql</code>	Dialecto Sequelize
<code>JWT_SECRET</code>	String	<code>KEY</code>	Clave secreta para firmar/verificar JWT (cambiar en producción)

⚠ **ADVERTENCIA:** Nunca usar `JWT_SECRET=KEY` en producción. Generar con: `node -e`

```
"console.log(require('crypto').randomBytes(64).toString('hex'))"
```

4. Autenticación y Autorización

4.1 Flujo de Autenticación JWT

```
Cliente → POST /api/v1/login { email, password }
```

```
Buscar usuario en BD
↓
Comparar password con bcrypt
↓
Firmar JWT { email, user_id } expiresIn: '1h'
↓
Cliente ← { ok: true, token: "eyJ..." }
```

4.2 Uso del Token en Peticiones Protegidas

```
GET /api/v1/cuentas
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

4.3 Middleware de Autenticación (`auth.middleware.js`)

```
// Extrae el token del header Authorization
// Verifica con jwt.verify(token, JWT_SECRET)
// Inyecta req.user = { email, user_id } si es válido
// Devuelve 401 si el token falta o es inválido
// Devuelve 403 si el token expiró
```

4.4 Ciclo de Vida del Token

- **Expiración:** 1 hora desde la emisión
- **Renovación:** El cliente debe hacer login nuevamente
- **Payload:** `{ email: string, user_id: UUID }`

5. Rutas y Endpoints de la API

Base URL: `http://[host]:3001/api/v1`

5.1 Módulo de Usuarios

Método	Ruta	Auth	Descripción
--------	------	------	-------------

POST	/user	❌ Pública	Registrar nuevo usuario
POST	/login	❌ Pública	Iniciar sesión + obtener JWT
GET	/user	✅ JWT	Obtener usuario autenticado
GET	/user/:id	✅ JWT	Obtener usuario por ID
PUT	/user/:id	✅ JWT	Actualizar usuario
DELETE	/user/:id	✅ JWT	Eliminar usuario (cascade)
POST	/user/initialize-categories	✅ JWT	Inicializar categorías predefinidas

Ejemplo – Registro:

```

POST /api/v1/user
Content-Type: application/json

{ "user_user": "correo@ejemplo.com", "user_password": "MiPass123" }

→ 200: { "ok": true, "id": "uuid", "token": "eyJ..." }
```

Ejemplo – Login:

```

POST /api/v1/login
Content-Type: application/json

{ "email": "correo@ejemplo.com", "password": "MiPass123" }

→ 200: { "ok": true, "token": "eyJ...", "id": "uuid" }
```

5.2 Módulo de Cuentas

Método	Ruta	Auth	Descripción
POST	/cuentas	✅ JWT	Crear cuenta financiera
GET	/cuentas	✅ JWT	Listar cuentas del usuario

GET	/cuentas/:id	✓ JWT	Obtener cuenta por ID
PUT	/cuentas/:id	✓ JWT	Actualizar cuenta
DELETE	/cuentas/:id	✓ JWT	Eliminar cuenta

Campos requeridos (POST): `account_name` , `account_type`

Campos opcionales: `account_balance` (default: 0), `account_currency` (default: 'COP')

5.3 Módulo de Categorías

Método	Ruta	Auth	Descripción
POST	/categorias	✓ JWT	Crear categoría personalizada
GET	/categorias	✓ JWT	Listar categorías del usuario
GET	/categorias/:id	✓ JWT	Obtener categoría por ID
PUT	/categorias/:id	✓ JWT	Actualizar categoría (no predefinidas)
DELETE	/categorias/:id	✓ JWT	Eliminar categoría (no predefinidas)

Nota: Las categorías con `category_is_predefined = 1` no pueden ser editadas ni eliminadas.

5.4 Módulo de Presupuestos

Método	Ruta	Auth	Descripción
POST	/presupuesto	✓ JWT	Crear presupuesto
GET	/presupuesto	✓ JWT	Listar presupuestos del usuario
GET	/presupuesto/:id	✓ JWT	Obtener presupuesto por ID
PUT	/presupuesto/:id	✓ JWT	Actualizar presupuesto

DELETE	/presupuesto/:id	✓ JWT	Eliminar presupuesto
--------	------------------	-------	----------------------

Campos: `budget_name` , `budget_amount` , `budget_currency` (default: 'COP')

5.5 Módulo de Gastos

Método	Ruta	Auth	Descripción
POST	/gastos	✓ JWT	Registrar gasto (descuenta de cuenta)
GET	/gastos	✓ JWT	Listar gastos del usuario
GET	/gastos/:id	✓ JWT	Obtener gasto por ID
PUT	/gastos/:id	✓ JWT	Actualizar gasto
DELETE	/gastos/:id	✓ JWT	Eliminar gasto

Campos requeridos: `expense_name` , `expense_amount` , `category_FK` , `account_FK`

Campos opcionales: `expense_description` , `expense_date` , `budget_FK`

Efecto: Al crear un gasto, se descuenta `expense_amount` del saldo de la cuenta (`account_FK`).

5.6 Módulo de Ingresos

Método	Ruta	Auth	Descripción
POST	/ingresos	✓ JWT	Registrar ingreso (suma a cuenta)
GET	/ingresos	✓ JWT	Listar ingresos del usuario
GET	/ingresos/:id	✓ JWT	Obtener ingreso por ID
PUT	/ingresos/:id	✓ JWT	Actualizar ingreso
DELETE	/ingresos/:id	✓ JWT	Eliminar ingreso

Efecto: Al crear un ingreso, se suma `income_amount` al saldo de la cuenta.

Asignaciones: Un ingreso puede repartirse entre presupuestos, metas, deudas y categorías vía `income_assignments`.

5.7 Módulo de Metas

Método	Ruta	Auth	Descripción
POST	/metas	✓ JWT	Crear meta financiera
GET	/metas	✓ JWT	Listar metas del usuario
GET	/metas/:id	✓ JWT	Obtener meta por ID
PUT	/metas/:id	✓ JWT	Actualizar meta
DELETE	/metas/:id	✓ JWT	Eliminar meta

Campos: `goal_name`, `goal_amount`, `goal_target_date`, `goal_description`

5.8 Módulo de Deudas

Método	Ruta	Auth	Descripción
POST	/deuda	✓ JWT	Registrar deuda
GET	/deuda	✓ JWT	Listar deudas del usuario
GET	/deuda/:id	✓ JWT	Obtener deuda por ID
PUT	/deuda/:id	✓ JWT	Actualizar deuda
DELETE	/deuda/:id	✓ JWT	Eliminar deuda

Campos: `debt_name`, `debt_amount`, `debt_date`, `debt_status` ('pendiente'), `debt_currency`

5.9 Módulo de Pagos de Deuda

Método	Ruta	Auth	Descripción
POST	/pagos-deuda	✓ JWT	Registrar pago de deuda
GET	/pagos-deuda	✓ JWT	Listar pagos de deudas
GET	/pagos-deuda/:id	✓ JWT	Obtener pago por ID
PUT	/pagos-deuda/:id	✓ JWT	Actualizar pago
DELETE	/pagos-deuda/:id	✓ JWT	Eliminar pago

6. Respuestas de Error Estándar

Código HTTP	Mensaje	Causa
400	Email and password are required	Campos faltantes en login
401	No autorizado / Token requerido	JWT ausente o inválido
403	No tienes permiso para acceder	Token expirado o acceso a recurso ajeno
404	Endpoint not found	Ruta no definida
404	Usuario no encontrado	ID no existe en BD
500	something went wrong in the request	Error interno del servidor

7. Modelos de Datos

7.1 Relaciones entre Modelos

```

User (1) ————— (*) Account
User (1) ————— (*) Category
User (1) ————— (*) Budget
User (1) ————— (*) Expense
User (1) ————— (*) Income
User (1) ————— (*) Goal
User (1) ————— (*) Debt

Expense (*) ——— (1) Category
Expense (*) ——— (1) Account
Expense (*) ——— (1) Budget [nullable]

Income (*) ——— (1) Account
Income (*) ——— (1) Category [nullable]
Income (1) ——— (*) IncomeAssignment

Debt (1) ——— (*) DebtPayment

```

7.2 Descripción de Campos Clave

users

- `user_id` : UUID CHAR(36), PK
- `user_user` : Correo electrónico (único)
- `user_password` : Hash bcrypt

income_assignments

- `assignment_type` : ENUM('debt', 'budget', 'goal', 'category')
- `target_id` : ID del elemento al que se asigna el ingreso

8. Frontend Web – Estructura Técnica

8.1 Árbol del Proyecto React

```

front-end-web/finance-frontend-react/src/
├─ App.jsx           # Router principal
├─ main.jsx          # Punto de entrada
├─ api/              # Configuración de axios/fetch
├─ components/       # Componentes reutilizables (16 componentes)
├─ contexts/         # Contextos React (Auth, etc.)
├─ hooks/            # Custom hooks
├─ pages/            # Vista por módulo (19 páginas)

```

```

|   | Login, Register
|   | Dashboard/Home
|   | Accounts, Categories
|   | Budgets, Expenses
|   | Incomes, Goals, Debts
|   | DebtPayments
|   └─ router/                # Definición de rutas React Router
|   └─ services/api/          # Capa de servicio HTTP (9 módulos)
|   └─ styles/                # CSS global + Tailwind
|   └─ utils/                 # Utilidades y helpers (4 archivos)

```

8.2 Variable de Entorno

```

# .env
VITE_API_URL=http://localhost:3001/api/v1

```

9. App Mobile – Estructura Técnica

9.1 Árbol del Proyecto Flutter

```

front-end-mobile/application_login/lib/
├─ main.dart                # Punto de entrada
├─ pages/
|   └─ auth/ (login.dart, register.dart)
|   └─ home/
|   └─ accounts/
|   └─ categories/
|   └─ budgets/
|   └─ expenses/
|   └─ incomes/
|   └─ goals/
|   └─ debts/
|   └─ more/
|   └─ splash/
├─ services/                # Capa de llamadas HTTP a la API
├─ theme/                   # Colores, tipografías, temas
└─ widgets/                 # Widgets reutilizables

```

9.2 Dependencias Flutter (pubspec.yaml)

```
dependencies:
  flutter: { sdk: flutter }
  cupertino_icons: ^1.0.8
  http: ^1.2.0 # Peticiones HTTP a la API
  google_fonts: ^6.2.1 # Tipografías de Google

environment:
  sdk: ^3.11.1
```

10. Errores Comunes y Diagnóstico

Síntoma	Causa	Diagnóstico	Solución
API sin respuesta	PM2 caído	<code>pm2 status</code>	<code>pm2 restart afip-api</code>
<code>ER_NO_DB_ERROR</code>	BD no creada	Verificar <code>.env</code> <code>DB_NAME</code>	Ejecutar <code>create_database.sql</code>
<code>401 Unauthorized</code>	Token ausente/inválido	Revisar header <code>Authorization</code>	Login para obtener nuevo token
<code>403 Forbidden</code>	Token expirado	Ver fecha de emisión del JWT	Login nuevamente
CORS error en navegador	CORS no configurado	Ver <code>app.js</code>	Verificar <code>app.use(cors())</code>
Flutter: <code>SocketException</code>	IP/URL incorrecta	Revisar <code>baseUrl</code> en services	Cambiar a IP de red local